NeuroCore

Nasıl Çalışır?

Mimari · Algoritmalar · Ölçümler

Sürüm 0.1.0 · 28.10.2026 · Windows / Python 3.10+

Bu kitapçık, sistemin nasıl çalıştığını **koddan doğrulanmış** algoritmalar ve **ölçülmüş** değerlerle açıklar.

İçindekiler

1. Yönetici Özeti
2. Sistem Mimarisi
3. Ajan Döngüsü Algoritması
4. Düşünme Döngüsü
5. Araç Sistemi
6. Kaynak Gözcüsü
7. Nöral Model
8. Eğitim Algoritması
9. Öğrenme Döngüsü ve Kalite Kuralları
10. Kasa (Bilgi Deposu)
11. Ortak Havuz (Çok Makineli)
12. Veri Akışı Diyagramları
13. Yapılandırma
14. Ölçülen Değerler
15. Sorun Giderme
16. Sözlük
Ek A. Dosya Haritası

1. Yönetici Özeti

NeuroCore; kaynakların yalnız boşta kalan kısmını kullanan, araştırma yapabilen, kod yazabilen, öğrendiklerini bir kasada tutan ve farklı makinelerde çalışan kopyalarıyla iş paylaşabilen yerel bir yapay zekâ ajanıdır. Üç ayrı zekâ katmanı vardır:

- Bulut dil modeli — planlama, araç seçimi ve cevap metni (ajanın "beyni").
- Yerel nöral model (211M parametre) — aday sıralama/seçim, eğitim ve bazı çeviri işleri.
- Kural tabanlı yol — bulut modeli yokken basit görevleri LLM'siz çözer.

Ölçülen temel değerler:

Ölçü	Değer	Nerede
Çekirdek araç	28	CORE_TOOLS
Uzaktan izinli araç	16 (yalnız okuma)	IZINLI_ARACLAR
Test paketi	1001 OK (tam) · 106 (öğrenme)	tests/
Nöral model	211.255.328 parametre	/api/learning
Eğitim geçmişi	15.904 adım · 107,46M token	growth.json
Tünel açılışı	6,3 sn	havuz_uctan_uca
Relay iş gecikmesi	1,7–2,7 sn	ajan_is_paylasimi
Havuz TTL / heartbeat	90 sn / 30 sn	peer_pool.py

2. Sistem Mimarisi

Sistem katmanlıdır. Her katman `initialize(config, event_bus)` (idempotent), `shutdown()` ve `get_stats()` sözleşmesini uygular; katmanlar olay yolu (event bus) üzerinden gevşek bağlıdır.

Katman	Dizin	Sorumluluk
L0 Çekirdek	neurocore/core	config, olay yolu, motor, gözcü
L1 Kaynak	core/governor.py	CPU/RAM/GPU bütçesi, durma/başlama
L2 Bilgi	neurocore/knowledge	kasa, SQLite indeks, bilgi grafı
L3 Nöral	neurocore/neural	model, eğitici, tokenizer, MoE
L4 Çok-modlu	neurocore/vision	ekran, görüntü, ses
L5 İşletim	neurocore/os	dosya, süreç, sandbox
L6 Dil	neurocore/language	algılama, çeviri, terim
L7 Planlayıcı	neurocore/planner	görev grafiği, checkpoint
Ajan	neurocore/agent	döngü, araçlar, düşünme, öğrenme
Ağ	neurocore/net	ortak havuz (keşif + iş paylaşımı)
Uygulama	neurocore/app	FastAPI sunucu, arayüz, launcher

2.1 Açılış sırası

```
launcher.setup()
  ├─ 1. çalışma alanı + .neurocore dizini
  ├─ 2. çekirdek (config, event bus)
  ├─ 3. kaynak gözcüsü     → anlık CPU/RAM/GPU ölçümü
```

```
  ├─ 4. kasa (vault)       → SQLite indeks
  ├─ 5. araç kayıt defteri → ToolRegistry() + coding/shell/vision/Blender
  ├─ 6. yerel sıralama     → LocalRankController (tembel model)
  ├─ 7. düşünme döngüsü    → ThinkingLoop (ranker'a bağlanır)
  ├─ 8. ajan köprüsü       → AgentBridge (parçalı iş)
  ├─ 9. dil modeli         → LLMRouter (deepseek/gemini/groq)
  ├─10. ajan döngüsü       → AgentLoop.initialize(registry, router)
  │      └─ attach_thinker(...)   ← planı yerel model seçer
  └─11. sunucu             → create_app(state) → uvicorn
         └─ _havuz_baslat() → ters tünel + kayıt + heartbeat
```

3. Ajan Döngüsü Algoritması

Görev, `AgentLoop.run()` ile yürütülür. İki mod vardır: `rules` (bulut modeli yok, kural tabanlı) ve `model` (araç çağrılı). Aşağıdaki sözde kod `model` modunun tam akışıdır.

```
GİRDİ: görev, max_steps=24, max_seconds=600, plan_first=False

1  messages ← [system, user(görev)]
2  plan_first ← plan_first ∨ (thinker bağlı ∧ |görev| ≥ 60)
3  EĞER plan_first:
4      adaylar ← bulut.tamamla("N farklı plan üret")        # tek çağrı
5      plan, meta ← thinker.think(görev, adaylar)           # yerel seçim
6      messages += [assistant(plan), user("Planı uygula")]
7      adım kaydı: kind="thinking"                          # denetim izi
8
9  deadline ← now + max_seconds
10 DÖNGÜ (adım ≤ max_steps ∧ now < deadline ∧ ¬iptal):
11     yanıt ← router.chat(messages, tools=select_tools(görev))
12     EĞER yanıt yok:
13         boş_sayısı += 1; EĞER boş_sayısı ≥ 3 → dur
14         devam
15     EĞER HTTP 413 ∧ araç şeması > 4:
16         şemaları yarıya indir; devam                      # istek küçült
17     EĞER HTTP 429:
18         bekle (20·deneme sn); deneme ≤ 3 ise tekrar
19     EĞER yanıt.tool_calls boş:
20         cevap ← yanıt.text; dur
21     HER araç çağrısı için:
22         ad ← düzelt(ad)                                   # shell_run → shell.run
23         EĞER (ad, args) son 3 adımda tekrarlandı → döngü kır
24         sonuç ← execute(ad, args)                         # bkz. adım 3.1
25         messages += [assistant(tool_call), tool(sonuç)]
26     adım += 1
27 ÖZET ← bulut.tamamla("yapılanları özetle") veya son metin
28 DÖNDÜR AgentRun(ok, adım[], araçlar[], süre, sağlayıcı)
```

3.1 Tek araç çağrısı: execute()

```
execute(ad, args):
1  EĞER ad ∈ IZINLI_ARACLAR ∧ havuz bağlı:                # 16 okuma aracı
2      yanıt ← havuz.komsu_dene({ad, args})               # önce KOMŞU
3      EĞER yanıt.ok → sonuç, kaynak ← (uzak sonuç, "relay:N")
4  EĞER sonuç yok:
5      sonuç ← registry.call(ad, args)                    # YEREL (onay dahil)
6      kaynak ← "yerel"
7  EĞER sonuç.needs_approval:                             # politika onay istedi
8      onay ← kullanıcıya sor(ad, risk, gerekçe)
9      EĞER ¬onay → dur (iptal)
10     registry.approve(ad); sonuç ← registry.call(ad, args)
11 adım kaydı: kind="tool", metin="[uzak:…]" veya "[yerel]"
12 DÖNDÜR sonuç
```

Parametreler (koddan):

Sabit	Değer	Anlam
max_steps	24	Bir görevde en fazla adım
max_seconds	600	Duvarsaati sınırı

REPEAT_LIMIT	3	Aynı araç+argüman tekrarı → döngü kır
MAX_EMPTY_RESPONSES	3	Boş yanıt sayısı sınırı
MAX_RATE_LIMIT_RETRIES	3	429 için yeniden deneme
RATE_LIMIT_WAIT_SECONDS	20	Bekleme tabanı (katları)
PLAN_CANDIDATES	3	Düşünmede aday plan sayısı
AUTO_PLAN_CHARS	60	Otomatik plan eşiği

4. Düşünme Döngüsü

Amaç: pahalı bulut modelinin ürettiği seçenekler arasından **yerel** modelin seçim yapması. Ölçüm: yerel model akıcı metin üretemiyor ama seçebiliyor (Blender kodunda %70, yaklaşımda %50; rastgele %25).

```
think(problem, propose, context):
1  adaylar ← propose(candidates = 3)          # bulut: tek çağrı, 3 plan
2  EĞER |adaylar| < 2:
3      DÖNDÜR (chosen=adaylar[0], source="cloud", sebep="tek_aday")
4  sıralama ← ranker.rank_candidates(problem, adaylar)
       # her aday için: skor = model.sequence_logprob(prompt + aday)
5  EĞER sıralama başarısız (yerel model yok):
6      DÖNDÜR (adaylar[0], source="cloud")   # güvenli geri düşüş
7  marj ← skor[0] − skor[1]
8  EĞER marj < min_margin (0.15):
9      DÖNDÜR (adaylar[0], source="cloud", güven="düşük")   # belirsiz
10 DÖNDÜR (adaylar[sıralama[0]], source="local", güven=marj)
```

Ölçülen örnek (bu makine): "Düşünme: 3 aday, kaynak=local, güven=orta, 18307ms" — sürenin çoğu yerel modelin tembel yüklenmesidir; model zaten yüklüyse sıralama 488 ms sürdü.

4.1 Neden yerel seçim?

- Ek LLM çağrısı yok: üç aday **aynı** yanıttan ayrıştırılır.
- Denetim izi: seçim `kind="thinking"` adımına yazılır (aday sayısı, kaynak, güven, marj).
- Güvenli geri düşüş: yerel model yoksa/hatalıysa ilk aday kullanılır ve bu açıkça not edilir.

5. Araç Sistemi

Araç, JSON şemalı bir fonksiyondur: ad, açıklama, parametreler, güvenlik seviyesi. `ToolRegistry` kayıt/politika/onay yönetimini yapar; `ToolResult` standart sonuçtur (ok, data, error, süre).

5.1 Araç seçimi algoritması (select_tools)

```
select_tools(görev, registry, max_tools=22):
1  seçili ← []
2  # (1) Göreve ÖZEL aileler önce → kesilirken düşmesinler
3  HER (anahtar_sözcükler, önekler) ∈ TOOL_HINTS:
4      EĞER görev içinde anahtar sözcük var:
5          seçili += registry[ad] : ad başlıyor(önek)
6  # (2) Çekirdek araçlar (her görevde)
7  seçili += CORE_TOOLS                       # 28 araç
8  # (3) Aile içinde YÜRÜTME araçları öne (alfabetik sona kalmasın)
9  seçili ← sırala(seçili, aile içi öncelik)
10 DÖNDÜR seçili[:max_tools]
```

Neden sınır? 83 aracın tamamı her istekte gönderilirse sağlayıcı isteği reddeder (HTTP 413) ve modelin doğru aracı seçmesi zorlaşır.

5.2 Güvenlik seviyeleri ve onay

Seviye	Örnek	readonly	supervised	trusted	yolo
READ	code.read_file, web.search	✓	✓	✓	✓
WRITE	code.edit_file, asset.uret	✗	onay	✓	✓
EXECUTE	shell.run, code.run_command	✗	onay	onay	✓
DANGEROUS	disk silme, süreç öldürme	✗	onay	onay	onay

Onay geldiğinde izin **tek kullanımlıktır** (consume=True): sonraki adımda yeniden sorulur. "Hatırla" seçilirse oturum boyunca geçerli olur. Onay reddedilirse ödül −0,3 (ceza değil, sinyal).

6. Kaynak Gözcüsü

En temel tasarım kuralı: ajan **ikinci sıradadır**. Kullanıcı ne kadar çok kullanıyorsa ajan o kadar küçülür.

```
harcanabilir = toplam × (1 − mevcut_kullanım) × 0.90
sert tavan   : RAM %85 · VRAM %85 · CPU %85
tabanlar     : CPU 1 çekirdek · RAM 128 MB · GPU 256 MB
histerezis   : cooldown_seconds boyunca bütçe büyümez
```

Sistem durumu	Ajanın kullanabileceği
CPU %50 kullanımda	toplamın %45'i
Sistem %90 boşta	toplamın %81'i (tavan %85)
RAM %85 kullanımda	toplamın %13,5'i
GPU başka süreçte	eğitim **başlamaz** (kaynaklar_dolu)

6.1 Plan küçültme sırası

```
plan_training(estimated_samples, steps, quality):
1  bütçe ← governor.budget()        # device, max_batch, max_ram_mb, max_gpu_mb
2  batch ← clamp(max_batch, 1, 32)
3  accum ← etkin_boyutu_koru(batch)  # küçük batch → birikim artar
4  adım  ← steps ∨ min(2000, örnek × epoch / (batch·accum))
5  lr    ← base × {3.0 | 1.5 | 0.6}  # kalite düşükse daha büyük
6  # doygunluk koruması (bu turda veri kaç kez görülecek)
7  görülme ← (adım · batch · accum · seq) / veri_token
8  lr ← base × {1.0 | 0.5 | 0.2 | 0.05}  göre görülme {<2 | ≥2 | ≥5 | ≥20}
9  amp  ← cihaz cuda                  # Pascal'da gerekirse kapatılır
10 DÖNDÜR TrainingPlan(batch, accum, seq, lr, adım, eval_every, save_every)
```

Ölçülen ders: doygunluk payında **ömür boyu** token sayacı kullanıldığında `görülme = 50,5` çıkıyor ve öğrenme oranı kalıcı olarak 5e-06'ya (30× düşük) sabitleniyordu. Pay artık **bu turun** hacmi / **bu veri kümesi** olarak hesaplanır.

7. Nöral Model

Bileşen	Değer
Parametre	211.255.328 (~211M)
Katman	8 · gömme 768 · kafa 12 · başlık boyutu 64
Sözlük	4.679 token (BPE)
MoE	5 uzman (temel 4 + 1 büyüme), top-2
Dizi uzunluğu	2048 (eğitimde 128–256)
Bağlılık	tie_embeddings = true · RoPE taban 10.000
Hassasiyet	fp32 (806 MB) · Adam ile 2.418 MB
Nesil	1 (uzman büyümesi 1, katman büyümesi 0)

7.1 Dosyalar

```
models/neurocore/model.pt        806 MB   ağırlıklar (fp32)
models/neurocore/config.json              mimari + büyüme kopyası
models/neurocore/growth.json              adım/token/uzman kaydı
models/neurocore/tokenizer.json           BPE sözlüğü (4.679)
data/checkpoints/neural/latest.pt      2,4 GB  ağırlık + Adam durumu
data/checkpoints/neural/step_*.pt      806 MB  her 500–750 adımda
```

Yeteneğin ölçülmüş durumu: model **tanıma** işlerinde işe yarıyor (aday seçimi %50–70), **üretimde** zayıf (kelime isabeti %23,1, anahtar terim 0/8). Sebep: 211M parametreye karşı 107,5M token = 0,51 token/parametre (Chinchilla ölçeğinin %2,5'i).

8. Eğitim Algoritması

```
train(model, dataset, plan, validation):
1  başlangıç ← evaluate(validation)                    # ilk kayıp
2  DÖNGÜ adım 1..plan.steps:
3      batch ← dataset.örnekle(plan.batch_size)
4      kayıp ← model(batch).loss + aux_loss            # MoE denge kaybı
5      EĞER amp: kayıp ← scaler.scale(kayıp)
6      kayıp.backward()
7      EĞER amp: scaler.unscale_(opt); clip_grad(1.0); scaler.step; scaler.update
8      EĞER adım % plan.eval_every == 0:
9          doğrulama ← evaluate(validation)
10         EĞER doğrulama < en_iyi: en_iyi ← doğrulama; ağırlıkları sakla
11         kaydet(step_{adım}.pt)                     # save_every
12     EĞER adım % 20 == 0: en_iyi_ağırlık_deepcopy()  # her adımda değil
13     EĞER bütçe_doldu ∨ süre_geçti → dur
14 final ← evaluate(validation)
15 DÖNDÜR TrainingResult(initial, final, best, val_history, tokens)
```

Doğrulama kaybı **asıl ölçüdür**; eğitim kaybı tek başına yanıltır (ezber). Ölçülen iyileşme:

Koşu	Doğrulama kaybı	Değişim
v2 havuzu (1.566 adım)	2,6645 → 1,7704	−%33,6
Oturum toplamı	6,44 → 1,77	−%72,5
Talimat karışımı (300 adım)	belge +%18 / talimat −%18	takas
Bugünkü kasa koşusu (33 adım)	1,9663 → 1,4087	−%28,4

8.1 GPU kararlılığı (ölçülen)

Bu makinede sürücü, yoğun eğitimde GPU'yu sıfırlıyordu (`nvlddmkm` Olay 153 = TDR; `TdrDelay` tanımsız, varsayılan 2 sn). Aşılma yolu ölçüldü: `--batch 8 --seq 128 --amp-yok` ile 300 adım kesintisiz; batch 32'de ilk adımlarda CUDA `unknown error`.

9. Öğrenme Döngüsü ve Kalite Kuralları

9.1 Ödül algoritması

```
compute_reward(run, user_feedback, had_error, self_recovered):
  EĞER user_feedback verilmiş → onu kullan (kırp: −1..+1)
  EĞER iptal ∨ onay reddi          → −0.3
  EĞER başarılı:
      kendi hatasını düzelttiyse   → +0.85   (en değerli: hata→çözüm)
      dosya değiştirdiyse          → +0.80
      araç kullandıysa             → +0.70
      yalnız sohbet                → +0.30
  EĞER başarısız ∧ adım > 1        → +0.10   (kısmi ilerleme)
  DEĞİLSE                          → −0.60

öğretici mi?  ok ∧ ödül ≥ 0.6 ∧ (araç ∨ dosya)
```

9.2 Akış

```
görev biter
  → deneyim kaydı (experiences.jsonl)
  → ödül hesaplanır
  → EĞER öğretici:
        KURAL 1 süzgeci (kısa/onay görevi → not YOK)
        KURAL 2 başlık arındırma
        KURAL 3 güven kalibrasyonu
        kasa notu yazılır (kind=skill)
  → evrim katmanına gözlem
  → EĞER bekleyen ≥ 50 deneyim ∧ kaynak uygun:
        eğitim (en az 20 adım) → model güncellenir
```

9.3 Dört kalite kuralı (bu sürümde eklendi)

Kural	Ne yapar	Ölçülen gerekçe
1 · Not süzgeci	görev < 20 karakter veya onay ağırlığı ≥ %60 → not yazılmaz	22 notun 11'i "Çözüm: onay veriyorum" idi, güven 0.925–0.990
2 · Başlık	sistem istemi (SORU:, "Sen bir …") ayıklanır	not başlığı istemin kopyasıydı
3 · Güven	tek adım → ≤ 0.6; adım × araç başarısı ile kalibre	tek adımlık onaya 0.925 veriliyordu
4 · Eğitim	5 deneyim/2 adım → 50 deneyim/20 adım; kasa: kind ∈ {reference,qa}, güven ≥ 0.6	kasanın %48,5'i skill notuydu

10. Kasa (Bilgi Deposu)

Kasa, Obsidian uyumlu markdown notlarıdır. Ajan buradan okur, araştırma sonuçlarını ve çözümlerini buraya yazar.

```
knowledge_base/
  10_concepts/     kavram notları
  20_howto/        nasıl yapılır
  30_reference/    başvuru / API
  70_journal/      günlük
```

```
  _index/          SQLite arama indeksi (üretilir) + sürüm geçmişi

not üst bilgisi (frontmatter):
  id, title, kind (concept|reference|qa|skill), category,
  tags, confidence, importance, created, updated, version, sources
```

10.1 Arama ve sıralama

```
ara(sorgu):
1  SQLite FTS + anahtar sözcük adayları
2  skor ← metin benzerliği + 0.1·(aynı kind) + kategori ağırlığı
3  düşük güvenli (skill) notlar eğitimde kullanılmaz
4  DÖNDÜR ilk K not (ajan bağlamına girer)
```

11. Ortak Havuz (Çok Makineli)

Farklı ağlardaki NeuroCore'lar birbirini bulur ve iş paylaşır. İnternete port açmak gerekmez; NAT ters SSH tüneliyle aşılır.

```
açılış:    token ← sunucudan (yoksa ilk düğüm oluşturur, 600)
           tünel ← ssh -N -R <20.000+hash(node_id)%20.000>:127.0.0.1:port
           kayıt ← peers.jsonl'e {node_id, ad, ip, port, relay_port, caps}
           heartbeat thread (30 sn)

iş:       execute(ad) → komsu_dene → relay(ssh + sunucuda urllib) → sonuç
           yanıt yok → YEREL çalıştır (iş kaybolmaz)

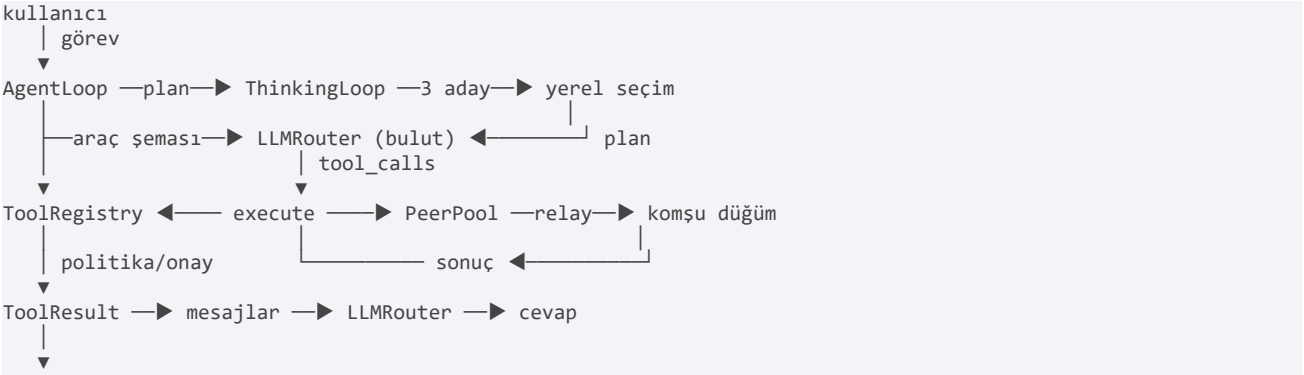
gözetim:  tünel süreci öldüyse yeniden aç (kayıt bayatlamasın)
kapanış:  kayıt silinir, tünel kapatılır
```

Öğe	Değer / kural
Kayıt TTL	90 sn
Heartbeat	30 sn
Relay portu	20.000 + (node_id[:4] taban16 % 20.000)
Uzaktan izinli araç	16 (yalnız okuma)
Kimlik doğrulama	paylaşılan küme anahtarı (sunucuda, 600)
Yasak	shell.run, code.run_command, dosya yazma, onay gerektiren

Ölçülen: tünel açılışı 6,3 sn · relay iş gecikmesi 1,7–2,7 sn · "Ters tünel yeniden açıldı" gözetimi gerçek çalışmada görüldü. Yerel devralma: komşu yoksa iş 2,7 sn içinde yerelde tamamlandı.

12. Veri Akışı Diyagramları

12.1 Görev akışı



```
LearningRecorder ──ödül──▶ kasa notu ──▶ (50 deneyim) eğitim
```

12.2 Eğitim akışı

```
kasa notları ─┐
deneyimler   ─┼─▶ süzgeç ─▶ TrainingDataset ─▶ plan_training ─▶ döngü
havuz        ─┘   (kind/conf)                  (batch/lr/adım)     │
                                                                   ▼
                              doğrulama kümesi ◀── evaluate ── checkpoint
                                     │
                                     ▼
                              kayıp eğrisi + en iyi ağırlıklar
```

13. Yapılandırma

Yol / değişken	İş
`.neurocore/peer.json`	küme sunucusu + SSH anahtar yolu (git dışı)
`NC_SERVER`, `NC_SSH_KEY`	aynı bilgiler, ortam değişkeni
`NC_PEER_TOKEN`	küme anahtarını elle sabitler
`NC_PEER=0`	ortak havuzu kapatır
`NC_EGITIM_NOTE_TURLERI`	eğitime girecek not türleri (varsayılan reference,qa)
`.neurocore/llm_credentials.json`	sağlayıcı API anahtarları
`.neurocore/logs/neurocore.log`	günlük (eğitim özetleri dahil)
`data/checkpoints/neural/`	checkpoint'ler

Masaüstü uygulaması seçenekleri:

```
python -m neurocore.app --workspace <dizin>
                        --autonomy readonly|supervised|trusted|yolo
                        --port 8000 --host 127.0.0.1
                        --no-window --browser --log-level info
```

14. Ölçülen Değerler

Konu	Ölçüm
Testler	1001 OK (tam paket, skipped=4)
Öğrenme testleri	106 OK (17 yeni kalite testi)
Kurulum doğrulaması	35/36 HAZIR (bf16 bilgi amaçlı)
Taşıma paketi	12,0 MB · 8.656 dosya
Model	211.255.328 parametre · 15.904 adım · 107,46M token
Eğitim hızı	~2,3 sn/adım (batch 32, seq 256) · 0,5–2,9k token/sn
Üretim kalitesi	%23,1 kelime isabeti · 0/8 anahtar terim
Yerel model payı	çağrı %0,6 · token %11,5 · araç yürütme %100
Havuz	tünel 6,3 sn · relay 1,7–2,7 sn · TTL 90 sn

15. Sorun Giderme

Belirti	Neden	Çözüm

`HAZIR` değil	eksik paket	`pip install -r requirements.txt`
CUDA `unknown error`	TDR / sürücü sıfırlaması	`--batch 8 --amp-yok` (kalıcı: TdrDelay artır)
`kaynaklar_dolu`	GPU/RAM dolu	bekle veya toplu işi küçült
Doğrulama kaybı artıyor	dağılım kayması / ezber	talimat ağırlığını düşür, veriyi çeşitlendir
Havuzda 1 düğüm	aynı `node_id`	`.neurocore/node_id` sil
`403 yetkisiz`	küme anahtarı yok/farklı	sunucudaki `cluster.token` ve yerel `peer_token`
ssh penceresi kırpışıyor	pythonw + alt süreç	sürüm 0.1.0+ (CREATE_NO_WINDOW uygulandı)
Not yazılmıyor	kalite kuralı 1/4	beklenen davranış

16. Sözlük

Terim	Tanım
Ajan döngüsü	Görevi plan → araç → gözlem döngüsüyle yürüten yürütücü (AgentLoop).
Araç (tool)	JSON şemalı, politika denetimli fonksiyon.
Düşünme döngüsü	Bulutun ürettiği adaylar arasından yerel modelin seçim yapması (ThinkingLoop).
Kasa	Markdown not deposu; ajanın uzun süreli bilgisi.
Checkpoint	Eğitim sırasında kaydedilen model anlık görüntüsü.
MoE	Mixture of Experts: her token yalnız birkaç uzmana gider.
TDR	Windows'un yanıt vermeyen GPU sürücüsünü sıfırlaması.
Relay	İşin sunucu üzerinden komşu düğüme taşınması.
Devralma (failover)	Komşu yanıt vermezse işi yerelin yapması.
Doğrulama kaybı	Görülmemiş verideki kayıp; asıl başarı ölçüsü.

Ek A. Dosya Haritası

```
neurocore/
  app/        launcher.py · server.py (FastAPI) · state.py
  agent/      loop.py · tools.py · thinking.py · local_rank.py
              learning.py · knowledge_work.py · bridge.py
  core/       engine.py · governor.py · config.py · work.py
  neural/     model.py · trainer.py · tokenizer.py · moe.py
  knowledge/  vault, layer, graph, index
  net/        peer_pool.py  (keşif + iş paylaşımı + tünel)
  planner/    görev grafiği ve yürütücü
  language/   algılama ve çeviri
tools/        ölçüm, eğitim, kurulum araçları
tests/        1001 test (ağ yok: SSH/LLM/HTTP sahtelenir)
docs/         bu kitapçık + kurulum/kullanım/havuz kılavuzları
```

Bu kitapçıktaki her sayı, bu makinede yapılan ölçümlerden veya koddan alınmıştır.
Sürüm 0.1.0 · github.com/termikprogram/neurocore